

HACKER ATTACK

SQL INJECTION



9

LISTA DE FIGURAS

Figura 1 – Exemplo de SQL Injection.....	5
Figura 2 – JDBC e JDBC Driver	6
Figura 3 – JDBC Driver Download	7
Figura 4 – Configurando um Java Project	8
Figura 5 – Criando uma pasta nova	9
Figura 6 – Estrutura de arquivos do Java Project.....	9
Figura 7– Mapeamento do JDBC Driver	10
Figura 8 – Mapeamento bem-sucedido do JDBC Driver	10
Figura 9 – Criação de uma Classe no Eclipse.....	11
Figura 10 – Consulta de Consumo.....	17
Figura 11 – Consulta de Consumo com SQL Injection.....	18

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Conexão JDBC no Java.....	11
Código-fonte 2 – Criação da Tabela TB_SMARTMETER	12
Código-fonte 3 – Exemplo de Comando INSERT no Java.....	13
Código-fonte 4 – Exemplo de Comando SELECT no Java	14
Código-fonte 5 – Três comandos INSERT	16
Código-fonte 6 – Código do Sistema de Consulta.....	17
Código-fonte 7 – Código Blindado para SQL Injection	19

SUMÁRIO

SQL INJECTION	4
1 INTRODUÇÃO	4
2 INJEÇÃO NO BANCO DE DADOS	4
3 INTEGRAÇÃO JAVA - SGBD	5
4 CONFIGURAÇÃO DO JDBC / JDBC DRIVER.....	6
5 CONFIGURAÇÃO DO JDBC NO JAVA	7
6 CONFIGURAÇÃO DA CONEXÃO JDBC COM O ORACLE	11
7 COMANDO INSERT	12
8 COMANDO SELECT	13
9 CONCEITO DE SQL INJECTION	15
10 LABORATÓRIO – SQL INJECTION	16
CONCLUSÃO.....	20
REFERÊNCIAS.....	21

SQL INJECTION

1 INTRODUÇÃO

Uma técnica comum que os atacantes utilizam para “bypassar” um sistema e chegar até o banco de dados, no qual o sistema está conectado, é o SQL Injection, e este ataque pode causar verdadeiros desastres dependendo da habilidade de SQL do atacante. Por isso, precisamos entender como funciona e como um sistema pode identificar essa vulnerabilidade e pará-la ou, ainda, como não permitir o *input* de qualquer injection.

2 INJEÇÃO NO BANCO DE DADOS

Anteriormente foi apresentado o *Code Injection* (injeção de código), que se trata de uma vulnerabilidade que pode estar presente em um sistema quando se utiliza a Programação Dinâmica sem o devido tratamento das informações provenientes de fora do sistema. No caso, é explorado apenas como injetar código externo (e malicioso) para ser executado pelo sistema.

As blindagens possíveis para esta categoria de ataques envolvem desde a remoção da componente de Programação Dinâmica do projeto, até a filtragem das informações externas.

Os ataques de injeção não se limitam apenas à linguagem utilizada para o desenvolvimento do sistema, é possível também explorar essa vulnerabilidade em comandos SQL no momento em que é feita a comunicação entre o sistema e o banco de dados (entre a camada *Controller* e o *Model*, vide modelo MVC de camadas). Essa classe de ataques é rotulada de *SQL Injection*, pois faz uso de comandos SQL não tratados para realizar o ataque nesse ponto de interação do sistema.

Antes de falarmos sobre *SQL Injection*, é necessário ter uma noção de como é feita essa integração entre o sistema e o SGBD (banco de dados), para que fique claro como uma análise cuidadosa e sistemática é importante para a detecção desse tipo de vulnerabilidade.



Figura 1 – Exemplo de SQL Injection
Fonte: Hackaday (2019)

Para isso, veremos como é feita a integração entre o SGBD e o sistema na linguagem Java, lembrando que essas integrações costumam apresentar diversas abordagens conforme a linguagem de programação utilizada, mas se utilizam dos mesmos princípios, que podem ou não impactar na forma com que o *SQL Injection* possa ser factível de ser explorado em um sistema. E também para que piadinhas envolvendo o *SQL Injection*, como na figura a seguir, possam ser compreendidas como sendo apenas uma piadinha de um mundo fictício.

3 INTEGRAÇÃO JAVA - SGBD

A integração entre o Java e o Banco de Dados (SGBD) pode ser realizada de diversas formas, desde a utilização de conectores nativos até o uso de bibliotecas (*frameworks*) de mapeamento automatizado.

Utilizaremos neste material os conectores nativos, pois neles é necessário explicitar os comandos SQL que serão enviados ao banco de dados e ao construir as *queries*, manualmente, é possível observar as potenciais vulnerabilidades, ao contrário do que no uso de *frameworks*.

A vantagem no uso de um *framework* de mapeamento reside no fato de as bibliotecas mais utilizadas (JPA e o Hibernate, por exemplo) possuírem algumas blindagens contra-ataques mais comuns, dentre elas o *SQL Injection*. No entanto, antes de se optar pelo uso de um *framework* desses, é necessário verificar a

documentação e nos catálogos de vulnerabilidades por eventuais brechas de segurança, pois no caso de existirem, será necessário no pior dos casos complementar o código do *framework* com blindagens adicionais.

Os conectores nativos do Java, via de regra, se utilizam de uma interface chamada de JDBC (**J**ava **D**ata**B**ase **C**onnectivity) que provê uma forma padronizada para possibilitar a conexão com o banco de dados por meio de um *driver* (chamado de JDBC Driver), servindo como um intermediário.

Uma ilustração de como a aplicação, o JDBC e o JDBC Driver atuam em alto nível na interação com o banco de dados é apresentada na figura a seguir.

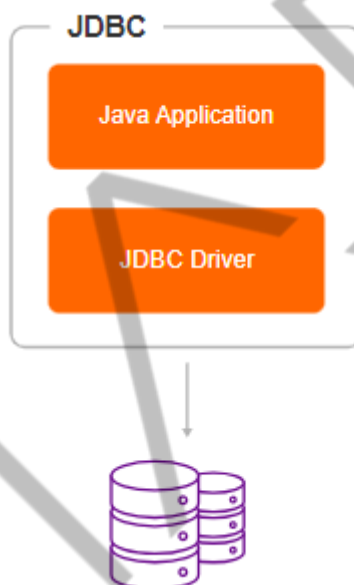


Figura 2 – JDBC e JDBC Driver
Fonte: Progress (2019)

Inicialmente trabalharemos na configuração de uma conexão JDBC e do JDBC Driver em uma aplicação Java que se comunicará com um banco de dados Oracle. Logo após a conexão ser validada e testada, veremos como enviar comandos SQL para realizar as operações de inserção, seleção, atualização e remoção de dados, na qual o Java possui certas peculiaridades na sua codificação.

4 CONFIGURAÇÃO DO JDBC / JDBC DRIVER

A configuração do JDBC começa pela obtenção de um JDBC Driver específico para o fornecedor do banco de dados utilizado no sistema, comumente

uma busca na web por “JDBC Driver <nome do banco de dados>” retorna a URL com os arquivos dos drivers.

Como utilizamos o banco de dados Oracle, buscaremos por este JDBC Driver específico, que também pode ser facilmente encontrado no site a seguir, lembrando que as URLs costumam mudar com frequência:

<<https://www.oracle.com/technetwork/database/application-development/jdbc/downloads/index.html>>

Busque pela versão do *driver* compatível com a versão do seu sistema gerenciador de banco de dados, de preferência evite baixar a última, por questões de maturidade do driver.

Aceite os termos de uso e logo abaixo uma listagem de arquivos relacionados ao JDBC Driver é apresenta, basta fazer o *download* do arquivo **ojdbc8.jar** (arquivos com extensão JAR são bibliotecas do Java), conforme ilustrado na figura a seguir:



Figura 3 – JDBC Driver Download
Fonte: Oracle (2019)

Ao finalizar o *download* do arquivo, guarde-o, pois usaremos em breve no momento de configurar o Java Project no Eclipse.

5 CONFIGURAÇÃO DO JDBC NO JAVA

Inicialmente, vamos criar um Java Project no Eclipse: **File > New > Java Project**. Colocaremos o nome do projeto como **JDBC** e depois é só clicar em **Finish**, conforme a figura a seguir:

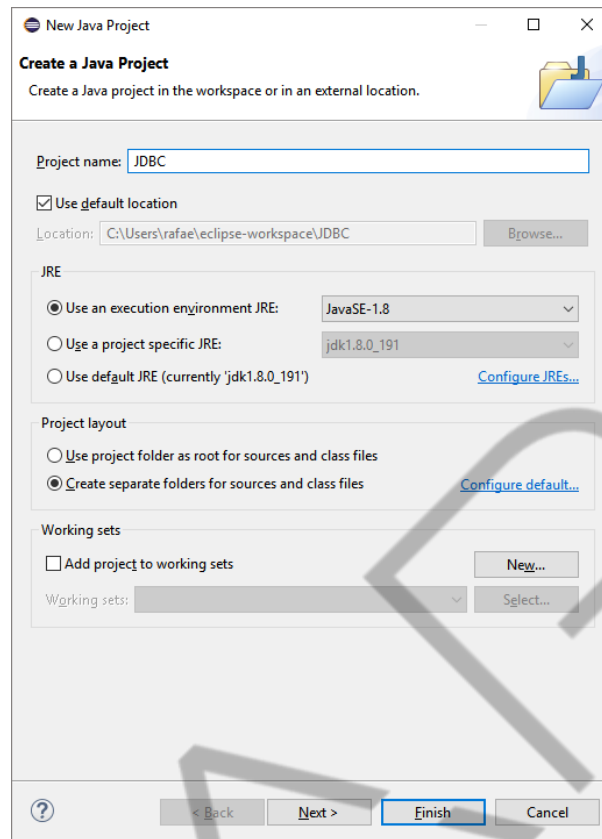


Figura 4 – Configurando um Java Project
Fonte: Elaborado pelo autor (2019)

Uma pasta com o nome do projeto será apresentada na parte esquerda da tela do Eclipse; dentro desta pasta, clique com o botão direito e vá em **New > Folder**, coloque o nome da pasta como **lib** e clique em **Finish**, conforme a figura a seguir:

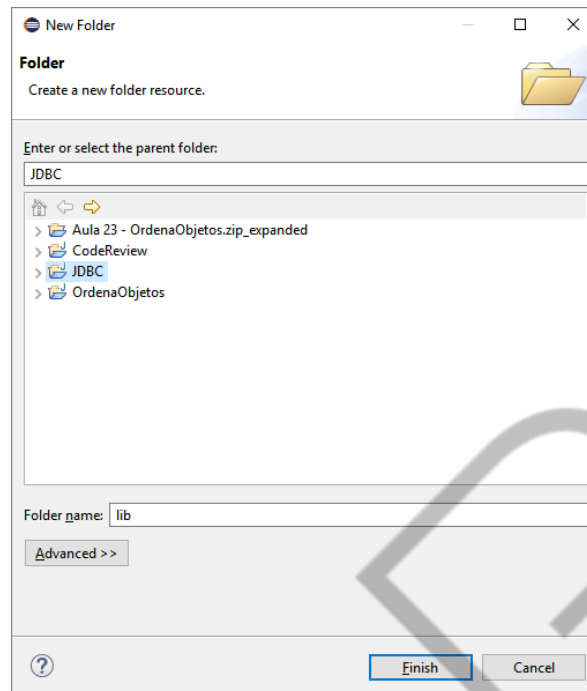


Figura 5 – Criando uma pasta nova
Fonte: Elaborado pelo autor (2019)

Dentro da pasta **lib** colocaremos o arquivo do JDBC Driver (**ojdbc8.jar**). Para isso, basta arrastar o arquivo para dentro da pasta **lib** e depois selecionar **Copy Files**. A estrutura de arquivos do Java Project ficará assim:

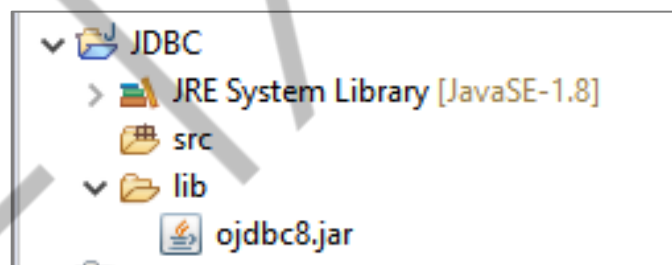


Figura 6 – Estrutura de arquivos do Java Project
Fonte: Elaborado pelo autor (2019)

Agora, basta configurarmos o projeto para reconhecer o JDBC Driver como uma biblioteca. Clique com o botão direito no projeto e vá em: **Build Path > Configure Build Path**, abrindo uma janela similar a essa:

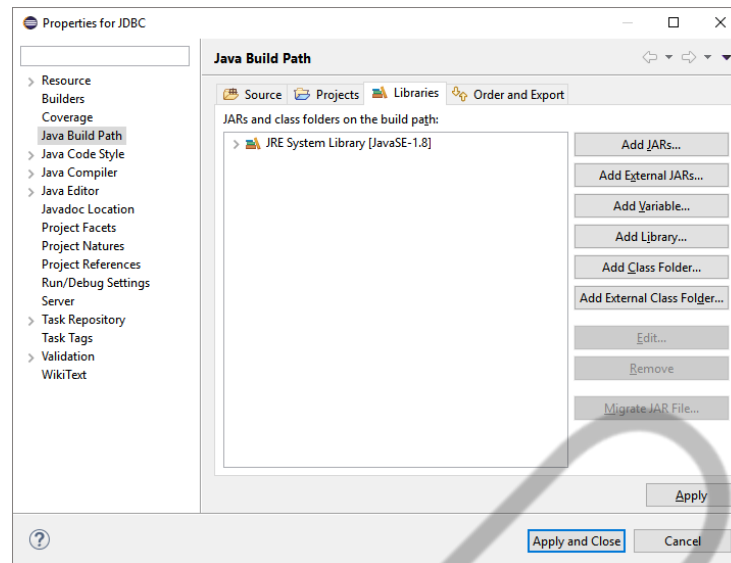


Figura 7– Mapeamento do JDBC Driver
Fonte: Elaborado pelo autor (2019)

Clique em **Add JARs...** (não confundir com o **Add External JARs...**) e selecione a biblioteca **ojdbc8.jar** que está dentro do projeto, depois clique em **Apply and Close**.

Se o procedimento foi executado corretamente, a biblioteca **ojdbc8.jar** terá no canto do ícone duas pequenas faixas de cor laranja e verde, conforme a figura a seguir:

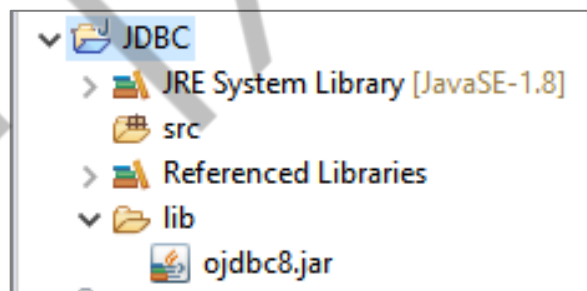


Figura 8 – Mapeamento bem-sucedido do JDBC Driver
Fonte: Elaborado pelo autor (2019)

As duas faixas indicam que o projeto está conseguindo localizar e mapear essa biblioteca para ser utilizada no projeto. Agora, vamos configurar a conexão JDBC.

6 CONFIGURAÇÃO DA CONEXÃO JDBC COM O ORACLE

Vamos criar uma classe no Java que conterá o código da conexão e das queries a serem utilizadas. Para isso, basta clicar com o botão direito no projeto e ir em: **New > Class**. Depois coloque no nome da classe **ConexaoTeste** e selecione a opção **public static void main(String[] args)**, conforme a figura a seguir:

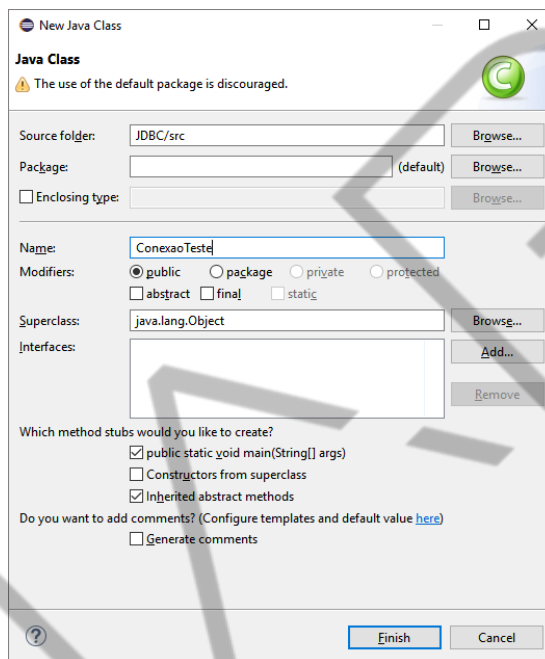


Figura 9 – Criação de uma Classe no Eclipse
Fonte: Elaborado pelo autor (2019)

Utilizemos o seguinte código nessa classe:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexaoTeste {
    public static void main(String[] args) throws
    SQLException {
        Connection conn = DriverManager.getConnection(
            "jdbc:oracle:thin:RM<numero>/<data
nasc>@oracle.fiap.com.br:1521:ORCL");
        System.out.println("Funcionou!");
        conn.close();
    }
}
```

Código-fonte 1 – Conexão JDBC no Java
Fonte: Elaborado pelo autor (2019)

Basta trocar o **<numero>** pelo número do seu RM e **<data nasc>** pela sua data de nascimento com o ano com dois dígitos (270518, por exemplo). Não retire a barra **"/"** que separa os dois campos.

Vamos executar esse código para avaliar se a conexão está funcionando. Clique no menu **Run** e depois em: **Run As... > Java Application**. Uma mensagem **Funcionou!** deverá aparecer no console, na parte inferior da tela, o que mostra o funcionamento da conexão.

Agora estamos preparados para enviar comandos SQL para o Oracle, para trabalhar isto, as próximas seções serão sobre como trabalhar os comandos básicos de manipulação de dados em uma tabela. Antes disso, vale lembrar a tabela TB_SMARTMETER utilizada anteriormente, que foi criada no SQL Developer por meio desse comando:

```
CREATE TABLE TB_SMARTMETER (
  NM_IDENT          VARCHAR2(200),
  NM_CLIENTE        VARCHAR2(200),
  VL_CARGA           NUMBER(9,2));
```

Código-fonte 2 – Criação da Tabela TB_SMARTMETER
Fonte: Elaborado pelo autor (2019)

Esta tabela será utilizada nos exemplos a seguir, então será necessário criá-la caso ainda não exista em seu banco de dados Oracle.

7 COMANDO INSERT

O seguinte código será utilizado neste exemplo, cujo funcionamento está detalhado a seguir:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class ConexaoTeste {
    public static void main(String[] args) throws
    SQLException {
        Connection conn = DriverManager.getConnection(
            "jdbc:oracle:thin:RM<numero>/<data
nasc>@oracle.fiap.com.br:1521:ORCL");
        PreparedStatement estrutura = null;
        estrutura = conn.prepareStatement("INSERT INTO
```

```

TB_SMARTMETER " + "(NM_IDENT, NM_CLIENTE, VL_CARGA) VALUES
(?, ?, ?)";
    estrutura.setString(1, "V701A");
    estrutura.setString(2, "Joaquim Afonso");
    estrutura.setDouble(3, 607);
    estrutura.execute();
    estrutura.close();
    conn.close();
}
}

```

Código-fonte 3 – Exemplo de Comando INSERT no Java
Fonte: Elaborado pelo autor (2019)

Esse código realiza a operação de INSERT de um registro no banco de dados, inicialmente estabelecendo a conexão, e depois o comando é preparado por meio de um objeto do tipo **PreparedStatement**, só não sendo explicitado os valores a serem inseridos.

Os valores utilizados no lugar das interrogações são preenchidos utilizando os métodos **setString** e **setDouble**, nos quais o primeiro parâmetro refere-se a qual interrogação será preenchido e o segundo parâmetro é o valor a ser preenchido.

Após o preenchimento dos valores, o método **execute()** executa o comando no banco de dados.

Como os objetos de conexão e de statement são fluxos inicializados, é necessário encerrá-los, sendo chamado o método **close()** para ambos (apesar de no encerramento da aplicação isso ocorrer naturalmente, em aplicações com disponibilidade elevada 24/7, é crucial fazer o encerramento de um recurso que não será mais utilizado ou que apenas será utilizado em um futuro remoto).

Esse comando INSERT também poderia ser construído apenas concatenando os valores, sem precisar utilizar os métodos **setString()** e **setDouble()**, esta forma veremos na parte prática.

8 COMANDO SELECT

O seguinte código será utilizado neste exemplo, cujo funcionamento será detalhado logo a seguir:

```

import java.sql.Connection;
import java.sql.DriverManager;

```

```
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ConexaoTeste {
    public static void main(String[] args) throws
    SQLException {
        Connection conn = DriverManager.getConnection(
            "jdbc:oracle:thin:RM<numero>/<data
nasc>@oracle.fiap.com.br:1521:ORCL");
        PreparedStatement estrutura = null;
        ResultSet resultado = null;
        estrutura = conn.prepareStatement("SELECT * FROM
TB_SMARTMETER");
        resultado = estrutura.executeQuery();

        while(resultado.next()) {
            System.out.println(resultado.getString("NM_IDENT"));
            System.out.println(resultado.getString("NM_CLIENTE"));
            System.out.println(resultado.getDouble("VL_CARGA"));
            System.out.println("---");
        }

        resultado.close();
        estrutura.close();
        conn.close();
    }
}
```

Código-fonte 4 – Exemplo de Comando SELECT no Java
Fonte: Elaborado pelo autor (2019)

A estrutura do código é semelhante ao do comando INSERT, visto na seção anterior, as diferenças principais estão na presença do objeto do tipo **ResultSet**, que guarda as informações que foram retornadas do banco de dados.

No caso de um SELECT, em vez do método **execute()**, é chamado o método **executeQuery()** no lugar.

O loop **while** está presente por conta da possibilidade de um **ResultSet** conter mais de um registro. Então, a cada laço, um registro é analisado e as informações são obtidas por meio dos métodos **getString()** e **getDouble()**, na qual deve-se informar o nome do campo em que deseja ser obtido o valor.

Os comandos UPDATE e DELETE seguem a mesma linha, com a diferença de que é chamado o método **executeUpdate()** no final da execução do UPDATE e **execute()** no final da execução do DELETE. Fica como exercício fazer as adaptações para esses dois cenários.

Agora, podemos falar finalmente de *SQL Injection* e de blindagens específicas para tratar o problema na parte prática.

9 CONCEITO DE SQL INJECTION

O *SQL Injection* é uma técnica que realiza, como o próprio nome diz, uma injeção de comandos SQL indevidos em um sistema. Assim como no *Code Injection*, essa entrada de comandos não é esperada, e quando não tratada, pode expor uma vulnerabilidade de sistema, permitindo com que o banco de dados seja acessado e manipulado indevidamente.

Isso ocorre, muitas vezes, quando o comando SQL é construído, pois um desenvolvedor pode optar por concatenar parte de um comando com informações providas do usuário, uma vez que a entrada pode conter lógica de programação que faz com que o comando final seja executado incluindo um processamento que não é esperado pelo sistema.

No exemplo da primeira figura deste tópico, está ilustrada uma maneira de explorar essa vulnerabilidade (que não se sabe se realmente funcionou), as câmeras de trânsito embutem um leitor de caracteres da placa de um carro para identificar o carro que poderá ser multado por violação de alguma lei de trânsito.

Mas vamos analisar o passo a passo desse processo. A câmera captura a imagem que depois é repassada para um *software* reconhecedor de caracteres e a partir da placa identificada, essas informações são cruzadas com uma base de dados de veículos.

Repare que o comando contido na placa está “incompleto”, a parte inicial do comando completa o que seria um comando normal em SQL, finalizando com o ponto e vírgula e logo depois acrescenta um **DROP DATABASE**.

Supondo que a câmera e o *software* consigam ter reconhecido o comando “incompleto” como um todo e construído o comando completo para executar no banco de dados, o comando SELECT (para consulta) é executado inicialmente e, logo depois, o banco de dados inteiro é deletado (**DROP DATABASE**).

Dependendo da maneira como a informação da placa do carro é repassada para um comando que será enviado para o banco de dados, é factível pensar que a base de dados de trânsito pode ter sido apagada completamente.

Agora temos o ferramental necessário para ver como funciona um *SQL Injection* na prática.

10 LABORATÓRIO – SQL INJECTION

Vamos trabalhar com a base de dados do *Smart Meter* utilizado anteriormente, com a tabela **TB_SMARTMETER** e os três registros na base de dados (ou quatro, se o exemplo do comando INSERT foi executado com sucesso).

Caso não tenha a tabela e nem os registros criados, execute no SQL Developer o Código-fonte “Criação da Tabela TB_SMARTMETER” e os comandos de inserção a seguir (os mesmos já executados):

```
INSERT INTO TB_SMARTMETER
(NM_IDENT, NM_CLIENTE, VL_CARGA)
VALUES
('A001T', 'Antonio Silva', 86.00);

INSERT INTO TB_SMARTMETER
(NM_IDENT, NM_CLIENTE, VL_CARGA)
VALUES
('N009Z', 'Maria Aparecida', 138.00);

INSERT INTO TB_SMARTMETER
(NM_IDENT, NM_CLIENTE, VL_CARGA)
VALUES
('W998A', 'Joao Diniz', 276.00);
```

Código-fonte 5 – Três comandos INSERT
Fonte: Elaborado pelo autor (2019)

Com a tabela **TB_SMARTMETER** e ao menos três registros, vamos supor que temos um sistema de consulta do valor de consumo de energia elétrica de um medidor inteligente (que representaria o consumo de uma residência, por exemplo).

Esse sistema basicamente pergunta o nome do cliente e cruza essas informações com a base de dados, retornando o valor do consumo atual do medidor desse cliente.

O código desse sistema é:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;

public class ConexaoTeste {
    public static void main(String[] args) throws
SQLException {
        Connection conn = DriverManager.getConnection(
            "jdbc:oracle:thin:RM<numero>/<data
nasc>@oracle.fiap.com.br:1521:ORCL");
        PreparedStatement estrutura = null;
        ResultSet resultado = null;
        String strCliente =
JOptionPane.showInputDialog("Digite o nome do cliente");
        estrutura = conn.prepareStatement("SELECT * FROM
TB_SMARTMETER WHERE NM_CLIENTE = '" + strCliente + "'");
        resultado = estrutura.executeQuery();

        while(resultado.next()) {
            System.out.println(resultado.getString("NM_CLIENTE"));
            System.out.println(resultado.getDouble("VL_CARGA") + "
KWh");
            System.out.println("---");
        }

        resultado.close();
        estrutura.close();
        conn.close();
    }
}
```

Código-fonte 6 – Código do Sistema de Consulta
Fonte: Elaborado pelo autor (2019)

O **JOptionPane** é um objeto que cria uma tela amigável para leitura de informações do usuário. Vamos informar um nome que está na base de dados, como **Joaquim Afonso**, e clique em **OK**.

Veja que obtemos o nome e o consumo do respectivo cliente:

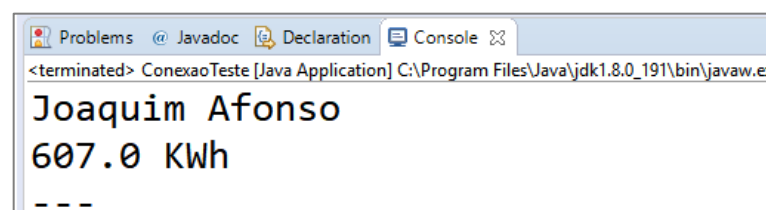
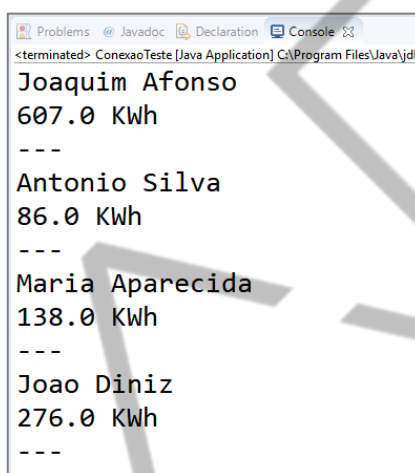


Figura 10 – Consulta de Consumo
Fonte: Elaborado pelo autor (2019)

Note que o comando SQL é construído com base em concatenação do valor informado pelo usuário com o restante do comando. Teste colocar nomes que estão e que não estão na base para ver o comportamento do sistema (que deverá ser coerente no sentido de que só deve exibir informações de clientes que constam na base de dados).

Vamos tentar forçar um *SQL Injection* nesse código, tente escrever o seguinte nome de cliente: **ZE' or '1' = '1** (sem fechar essa última aspa).

Qual foi o resultado? Inesperado, não?



```
<terminated> ConexaoTeste [Java Application] C:\Program Files\Java\jdk1
Joaquim Afonso
607.0 KWh
---
Antonio Silva
86.0 KWh
---
Maria Aparecida
138.0 KWh
---
Joao Diniz
276.0 KWh
---
```

Figura 11 – Consulta de Consumo com SQL Injection
Fonte: Elaborado pelo autor (2019)

Esse comando injetado faz com que o sistema imprima todos os resultados, por causa do seguinte: a parte **ZE'** termina a condição do WHERE referente ao nome do cliente a ser pesquisado (ele não contém aspas no início, pois já existem aspas sendo abertas antes da concatenação, observe no código).

Logo depois, entra o comando injetado, que é o **or '1' = '1**, essa segunda parte torna o WHERE sempre verdadeiro, como um todo, retornando todos os resultados da tabela.

É uma maneira muito criativa de injetar comandos SQL, há diversas outras formas (inclusive mais destrutivas).

Mas como evitar um ataque desses? Resolve usar as blindagens que vimos na parte de *Code Injection*? Sim, resolve. Mas como já vimos, é trabalhoso lembrar todas as possibilidades de palavras-chave que podem ser utilizadas em um ataque de injeção.

Existe uma maneira mais prática dentro do Java para evitar grande parte dos tipos de *SQL Injection*, basta utilizar os métodos **setString()** e variantes, como visto neste capítulo.

O código do nosso *Smart Meter* blindado consta a seguir:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;

public class ConexaoTeste {
    public static void main(String[] args) throws SQLException {
        Connection conn = DriverManager.getConnection(
            "jdbc:oracle:thin:RM<numero>/<data
nasc>@oracle.fiap.com.br:1521:ORCL");
        PreparedStatement estrutura = null;
        ResultSet resultado = null;
        String strCliente = JOptionPane.showInputDialog("Digite
o nome do cliente");
        estrutura = conn.prepareStatement("SELECT * FROM
TB_SMARTMETER WHERE NM_CLIENTE = ?");
        estrutura.setString(1, strCliente);
        resultado = estrutura.executeQuery();

        while(resultado.next()) {
            System.out.println(resultado.getString("NM_CLIENTE"));
            System.out.println(resultado.getDouble("VL_CARGA") + "
KWh");
            System.out.println("---");
        }

        resultado.close();
        estrutura.close();
        conn.close();
    }
}
```

Código-fonte 7 – Código Blindado para SQL Injection
Fonte: Elaborado pelo autor (2019)

A mudança basicamente consiste no uso do método **setString()** para inserir o valor do nome do cliente no comando SQL a ser executado.

Vale mencionar que o **setString()** realiza a sanitização dos dados, ou seja, não deixa executar caracteres especiais, aumentando a segurança na parte de validação das informações.

Colocando o mesmo nome de cliente com *SQL Injection* utilizado anteriormente: **ZE' or '1' = '1'**, é possível observar que não traz nenhum resultado, conforme esperado.

Um especialista mais atento tentaria uma variante dessa entrada, colocando as aspas no início e no final, para injetar um comando completo: **'ZE' or '1' = '1'**, mesmo assim não retorna nenhum resultado.

CONCLUSÃO

Ataques do tipo *SQL Injection* são mais difíceis de serem percebidos, pois o comando injetado passa por meio do sistema sem ser detectado, e sua manifestação ocorre apenas no momento em que o comando completo gerado (com o comando injetado) é executado no banco de dados.

Esse ataque possui as mais variadas formas, muitas delas se utilizam de caracteres especiais ou de lógica, tornando muito-se difícil construir blindagens baseado-se em um conjunto de palavras-chave, como no *Code Injection*.

No Java, é possível utilizar métodos específicos para construção de comandos SQL sanitizados, evitando boa parte das formas de *SQL Injection*, mas não evita todas as suas formas.

Vale lembrar que em sistemas modernos, muitas vezes, o comando SQL é gerado automaticamente, tornando necessário testar os *frameworks* de persistência contra formas variadas de *SQL Injection* para conseguir avaliar sua robustez contra este tipo de ataque, pois nem sempre esses *frameworks* têm isso discriminado em sua documentação.

É possível sequestrar informações de um usuário de outra forma que não utilizando *Code Injection* ou *SQL Injection*?

Sim, é possível. Uma dessas maneiras é fazendo o sequestro da sessão (ou dos dados de login) do usuário, enquanto estiver navegando em um *website*, situação esta que trataremos em detalhes posteriormente.

REFERÊNCIAS

HACKADAY. **SQL Injection Fools Speed Traps and Clears your Record.** 2014. Disponível em: <<https://hackaday.com/2014/04/04/sql-injection-fools-speed-traps-and-clears-your-record/>>. Acesso em: 20 nov. 2019.

ORACLE. **Oracle Database 18c (18.3) JDBC Driver & UCP Downloads** Disponível em: <<https://www.oracle.com/technetwork/database/application-development/jdbc/downloads/jdbc-ucp-183-5013470.html>>. Acesso em: 20 nov. 2019.

PROGRESS. **How Does JDBC Work?** Disponível em: <<https://www.progress.com/fags/datadirect-jdbc-fags/how-does-jdbc-work>>. Acesso em: 20 nov. 2019.